Tup — Raspberry Pi Deployment Guide

End-to-end steps to host the Tup app stack on a Raspberry Pi 4 and expose it publicly via a Cloudflare Tunnel and a Cloudflare-registered domain, with automatic HTTPS terminated at Cloudflare's edge.

> **Update:** this guide previously used DuckDNS + router port-forwarding for ports 80/443. That approach was abandoned because this ISP assigns the router a Carrier-Grade NAT address (RFC 6598, `100.64.0.0/10`) on its WAN interface — there is no real public IP to forward ports to, so inbound traffic from the internet can never reach the router no matter how port forwarding is configured. The fix is an outbound-only Cloudflare Tunnel, which works regardless of CGNAT since the Pi initiates the connection to Cloudflare instead of waiting for inbound connections.

1. Network Prerequisites

1 Find the router and the Pi on the local network

```
# Find the router (default gateway)
ip route | grep default

# Test connectivity to the Pi (example IP)
ping -c 3 -W 2 192.168.1.132
```

2 Reserve a fixed local IP for the Pi

In the router admin panel (e.g. `http://192.168.1.1`), open **DHCP Reservation / Address Reservation** and bind the Pi's MAC address to a fixed IP:

Field	Example value
MAC Address	DC:A6:32:DE:B1:89
Reserved IP	192.168.1.132

3 Port forwarding is NOT required

> Unlike a traditional port-forwarding setup, Cloudflare Tunnel opens an **outbound-only** connection from the Pi to Cloudflare's edge, so no router configuration (port forwarding, DMZ, virtual servers) is needed at all — this also works even if the ISP assigns a CGNAT address and there is no real public IP, which is the case here (confirmed WAN IP in the `100.64.0.0/10` range).

2. SSH Access to the Pi

4 Set up passwordless SSH (key-based auth)

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub cristi@192.168.1.132

# Verify
ssh -o BatchMode=yes -o ConnectTimeout=5 cristi@192.168.1.132 "echo CONNECTED"
```

> Never share your SSH password in plain text (chat, tickets, etc.). Prefer key-based auth and rotate any password that was exposed.

3. Resolve Port Conflicts (Pi-hole)

If the Pi already runs Pi-hole, its web UI binds ports 80/443, which conflicts with the Caddy reverse proxy. Move Pi-hole off those ports:

```
# Check what is listening on 80/443
sudo ss -tlnp | grep -E ':80 |:443 '

# Edit the Pi-hole webserver port in /etc/pihole/pihole.toml
# from:   port = "80o,443os,[::]:80o,[::]:443os"
# to:     port = "8080o,8443os,[::]:8080o,[::]:8443os"
sudo sed -i 's/port = "80o,443os,\[::\]:80o,\[::\]:443os"/port = "8080o,8443os,[::]:8

sudo systemctl restart pihole-FTL
```

4. Install Docker on the Pi

```
curl -fsSL https://get.docker.com -o /tmp/get-docker.sh
sudo sh /tmp/get-docker.sh
sudo usermod -aG docker cristi

docker --version
docker compose version
```

> On some Raspberry Pi OS images, `uname -m` reports `aarch64` even though Docker runs a 32-bit (`arm/v7`) userland. If image pulls fail with "no matching manifest", force the platform explicitly:

```
DOCKER_DEFAULT_PLATFORM=linux/arm/v7 docker compose ...
```

Also verify any extra images (e.g. `linuxserver/duckdns`) actually publish an `arm/v7` variant before relying on them — some only ship `amd64`/`arm64`.

5. Copy the Project to the Pi

```
rsync -az --info=progress2 \
  --exclude 'target' \
  --exclude '.git' \
  --exclude 'node_modules' \
  --exclude '*/target' \
  ./ cristi@192.168.1.132:~/apps/tup/

# Promote to the final deploy path used by systemd
sudo mkdir -p /opt/tup
sudo rsync -a ~/apps/tup/ /opt/tup/
```

6. Cloudflare Tunnel — Domain + Outbound-Only Public Access

Register (or transfer) a domain via Cloudflare Registrar or add an existing domain to a Cloudflare account as a zone (e.g. `tup-app.dev`). The domain's zone **must** be in the same Cloudflare account that creates the tunnel, since Public Hostname routing auto-manages the DNS record. A free DuckDNS-style subdomain cannot be used here for that reason.

5 Create the tunnel and install `cloudflared` natively

In the Cloudflare Zero Trust dashboard, go to **Networks** → **Tunnels** → **Create a tunnel** (Cloudflared connector type) and copy the generated tunnel token.

> Do **not** run `cloudflared` as a Docker container on this Pi. The official `cloudflare/cloudflared` image only publishes `amd64`/`arm64` manifests — there is no 32-bit `arm/v7` build. This Pi's Docker daemon reports a 32-bit (`arm/v7`) userland even though the kernel is `aarch64`, so forcing the `arm64` image via `--platform` pulls successfully but fails to execute (exits silently with code 159, no logs) — the 32-bit container runtime cannot run 64-bit binaries. Install `cloudflared` natively via apt instead.

```
# Add Cloudflare's apt repo and install cloudflared natively (armhf package)
curl -fsSL https://pkg.cloudflare.com/cloudflare-main.gpg | sudo tee /usr/share/keyri
echo 'deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg] https://pkg.cloudflare.
  sudo tee /etc/apt/sources.list.d/cloudflared.list
sudo apt-get update && sudo apt-get install -y cloudflared

# Register cloudflared as a systemd service using the tunnel token from the dashboard
sudo cloudflared service install <CLOUDFLARE_TUNNEL_TOKEN>

# Confirm it's running and connected
sudo systemctl status cloudflared
sudo journalctl -u cloudflared -n 20
```

> Use the apt codename that actually matches what `apt-get update` resolves on this Pi (confirmed as `noble`, not `bookworm`, on the current Raspberry Pi OS image) — check `/etc/os-release` or the apt output if unsure.

6 Add the Public Hostname route

> This is a separate, easy-to-miss step from creating the tunnel. A tunnel can show "healthy"/connected with zero actual routing rules and zero DNS records until a Public Hostname is added.

In the tunnel's dashboard, open the **Public Hostname** tab and add a route:

Field	Value
Domain	tup-app.dev
Type	HTTP
URL	localhost:80

Because `cloudflared` runs natively on the host (not inside the Docker Compose network), the ingress target must be `localhost:80`, not a Docker service name like `caddy:80`. Adding this route also auto-creates the public DNS record for the domain, pointing at Cloudflare's anycast IPs.

> DNS propagation speed varies by resolver even once correctly configured — major public resolvers (1.1.1.1, 8.8.8.8, 9.9.9.9, 208.67.222.222) typically pick it up quickly, but an ISP's own resolver can lag behind by several minutes or more. To validate the app itself is working without waiting on DNS, bypass resolution entirely:

```
# Get a known-good IP from a fast public resolver
dig @1.1.1.1 +short tup-app.dev

# Hit the app directly via that IP, still over real TLS/SNI
curl --resolve tup-app.dev:443:<ip-from-above> https://tup-app.dev/
```

7. Google OAuth Redirect URI

In Google Cloud Console, add the production callback URL to the OAuth client's authorized redirect URIs:

```
https://tup-app.dev/auth/google/callback
```

8. Environment File

```
# /opt/tup/.env
PUBLIC_DOMAIN=tup-app.dev
GOOGLE_CLIENT_ID=<your-client-id>
GOOGLE_CLIENT_SECRET=<your-client-secret>
```

> `CLOUDFLARE_TUNNEL_TOKEN` is only needed once, for the `cloudflared service install` command in step 5 — it is not read by Docker Compose and does not need to go in this file.

9. Build and Launch the Stack

```
cd /opt/tup
sudo DOCKER_DEFAULT_PLATFORM=linux/arm/v7 \
  docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --no-build
```

Caddy now serves plain HTTP on port 80 only (published to the host as `80:80`) and no longer manages any TLS certificates — TLS is terminated at Cloudflare's edge, and traffic reaches Caddy through the Cloudflare Tunnel.

10. Auto-Start on Boot (systemd)

```
sudo cp deploy/systemd/tup.service /etc/systemd/system/tup.service
sudo systemctl daemon-reload
sudo systemctl enable --now tup.service

# Check status / logs
systemctl status tup.service
journalctl -u tup.service -f
```

> `cloudflared` is a separate systemd service (installed in step 5) that starts independently on boot — it is not managed by `tup.service` or Docker Compose.

11. Verify From Outside the Network

```
curl -I https://tup-app.dev
```

Since there are no inbound ports to check, a third-party port checker (e.g. canyouseeme.org) is not useful here — instead confirm the tunnel has active connections (`cloudflared tunnel info` or the dashboard's tunnel overview) and that the Public Hostname route is listed.

Architecture Overview

Component	Purpose
Cloudflare Tunnel (cloudflared, native systemd service)	Outbound-only connection from the Pi to Cloudflare's edge; works regardless of CGNAT; no inbound ports needed
Cloudflare edge	Terminates public TLS/HTTPS for tup-app.dev and forwards requests through the tunnel
Caddy	Reverse proxy on plain HTTP :80, routes `/auth/*` to auth-service and everything else to frontend-service
auth-service	Handles `/auth/*` routes, Google OAuth, JWT/session cookies
frontend-service	Serves the Leptos web app, proxies to backend/auth APIs
backend-service	Farmer registration API
db	PostgreSQL database